

OPEN TRACK

PAYMENT INFRASTRUCTURE

Executable State Architecture (ESA)

Policy-Bounded Multi-Agent Runtime for Adaptive Payment Infrastructure

Detailed Product Requirements Document

Buildathon / Prototype Edition | Prepared from the existing ESA PRD and adapted for a payment-infrastructure Open Track submission

Core proposition

Payment workloads can change faster than static infrastructure rules can safely adapt. ESA turns runtime state into active, policy-aware entities and uses specialized AI agents to recommend bounded infrastructure actions. A deterministic safety layer remains the final authority.

Document Control

Field	Value
Project	Executable State Architecture (ESA)
Prototype domain	Payment infrastructure adaptation and operational resilience
Track	Razorpay Buildathon - Open Track
Primary artifact	Working prototype + public repository + 5-minute pitch + architecture
Status	Detailed PRD for implementation
Primary constraint	Do not claim access to or control of Razorpay production infrastructure

1. Executive Summary

ESA is a policy-bounded autonomous runtime designed for payment infrastructure that must adapt to rapidly changing traffic, latency, availability, cost, and failure conditions. The system treats operational state as active and executable rather than passive data. Each state entity carries the information needed to influence placement, replication, lifecycle, security, and execution, while the runtime enforces deterministic safety boundaries.

For the payment-angle prototype, ESA consumes payment events and infrastructure telemetry, maintains a redacted payment-workload state model, and uses a small multi-agent orchestration layer to detect conditions, diagnose causes, and plan bounded actions. Proposed actions are expressed through typed contracts and are sent through a deterministic Policy Engine, Decision Verifier, and Action Gateway before any runtime mutation occurs.

The prototype is intentionally payment-infrastructure oriented but does not claim to reproduce or replace Razorpay internal systems. Razorpay Test Mode and official APIs/webhooks can be used as an integration surface for realistic payment events, while the adaptive runtime itself remains a controlled research prototype. Razorpay documents Test Mode as a sandbox for simulated transactions and supports payment webhooks such as payment.authorized, payment.captured, payment.failed, and order.paid. [See External Reference Notes]

Buildathon thesis

AI should not be granted unrestricted control over payment infrastructure. AI should propose bounded infrastructure actions from live state; deterministic controls should verify, modify, deny, execute, and audit those actions.

2. Problem Definition

2.1 Payment infrastructure problem

Payment systems operate under variable demand and strict correctness requirements. A traffic burst, regional skew, downstream latency increase, node failure, or event-processing backlog can change the operational state of the system within seconds. A runtime that reacts only through static rules may have difficulty handling combinations of signals, while an unrestricted AI operator creates unacceptable safety and auditability risks.

The target problem is therefore not payment fraud detection, payment recommendation, or revenue recovery. It is the infrastructure-control problem behind reliable payment processing: how to adapt compute, routing, replication, locality, and recovery behavior quickly while keeping every automated action bounded, explainable, auditable, and reversible.

2.2 Problem statement

How can payment infrastructure autonomously adapt to changing workloads, latency requirements, regional demand, and failures while ensuring every AI-driven scaling, routing, replication, migration, and recovery action remains policy-bounded, verifiable, auditable, and reversible?

2.3 Why Open Track

The Open Track is appropriate because ESA addresses an infrastructure-control problem rather than the primary business problems described by the predefined commerce, fraud/risk, revenue-recovery, or finance-controller tracks. The project can therefore demonstrate a distinctive systems contribution while still using payment workflows as the real-world operating context.

3. Product Vision

Create a runtime in which payment state and infrastructure state are continuously observed, represented as executable state entities, and adapted by specialized AI agents under a deterministic safety constitution.

Vision element	Target outcome
Adaptive	React to workload and failure conditions without waiting for manual intervention for low-risk actions.
Policy-bounded	No AI component can bypass hard rules, access controls, or typed action contracts.
Payment-aware	Prioritize availability, correctness, latency, and safe recovery for payment workloads.
Auditable	Every proposed, modified, denied, and executed action has decision lineage.
Reversible	Major topology mutations have rollback or compensating controls.
Measurable	Demonstrate improvements against a rule-only baseline using controlled workloads.

4. Goals and Non-Goals

4.1 Goals

- Build a working payment-infrastructure prototype driven by synthetic and Razorpay Test Mode payment events.
- Model payment workload state as executable, policy-aware state entities.
- Use 3-4 specialized agents for monitoring, diagnosis, planning, and bounded policy reasoning.
- Generate typed actions such as CREATE_REPLICA, SHIFT_ROUTE, MIGRATE_PARTITION, THROTTLE_WORKLOAD, and ROLLBACK.
- Enforce every action through deterministic policy checks and a single Action Gateway.
- Provide observability, action lineage, explanations, and rollback support.
- Benchmark adaptive orchestration against rule-only orchestration.
- Demonstrate at least one measurable improvement in latency, recovery time, or replica efficiency.

4.2 Non-goals

- Replace Razorpay or another payment gateway.
- Access or modify Razorpay production infrastructure.
- Process real card data or expose sensitive payment credentials to AI models.
- Build a full production-grade distributed database in the Buildathon prototype.
- Claim perfect security, zero downtime, or autonomous operation without limits.
- Use unrestricted shell, kubectl, SQL, or administrative execution by agents.
- Build every capability described in the long-term 12-month ESA roadmap before submission.

5. Users and Stakeholders

Persona	Need	How ESA helps
Payment platform engineer	Keep payment workloads available and responsive.	Adaptive runtime actions with guardrails and auditability.
SRE / infrastructure operator	Reduce manual diagnosis and operational toil.	Agents summarize signals, diagnose incidents, and propose bounded changes.
Security / risk operator	Prevent unsafe autonomous changes.	Policy Engine, risk scoring, signed actions, and human approval for high-risk actions.
Developer integrating payments	Express workload goals without low-level infrastructure commands.	Intent schema maps goals to runtime constraints.
Research / evaluation reviewer	See measurable technical value.	Benchmarks, ablations, audit logs, and reproducible workload traces.

6. Payment-Angle Product Concept

6.1 Payment workload model

The prototype treats payment processing as a sequence of state transitions and operational signals, not as raw sensitive financial data. The payment-facing state model should contain only the minimum metadata required for orchestration.

State field	Example	AI exposure
Event type	payment.failed	Allowed
Order/payment reference	PSEUDONYMIZED-1234	Allowed; no raw secrets
Region	IN-SOUTH	Allowed
Payment method class	UPI / CARD / NETBANKING	Allowed
Latency bucket	P95_>_500MS	Allowed
Workload rate	1200 events/min	Allowed
Failure ratio	0.082	Allowed
Credentials / card data	Never stored in prompt	Blocked

Privacy rule

No card number, CVV, secret, API key, authentication token, or unnecessary personally identifiable information is sent to the SLM or agent prompts. Agents reason over aggregated and redacted state summaries.

6.2 Primary operating scenarios

Scenario	Trigger	Target action	Expected evidence
Regional hotspot	P95 latency rises in one region while load becomes skewed.	Temporary replica + locality-aware routing.	Lower P95 latency and stable error rate.
Processing node failure	Heartbeat/health signal indicates a node failure.	Failover + replica promotion + workload rebalancing.	Recovery time and successful state preservation.
Webhook backlog	Payment event queue depth and processing latency exceed thresholds.	Scale worker pool or move workload to healthy capacity.	Queue drain time improves.
Cost pressure	Low traffic persists across a region/time window.	Reduce temporary capacity within safety bounds.	Cost proxy decreases without SLA violation.

7. Functional Requirements

FR-01 Event ingestion

The system shall ingest payment events from Razorpay Test Mode webhooks and/or a synthetic event generator. Razorpay documentation describes webhooks as server-to-server event notifications and recommends server-side webhook handling for payment state changes. Test Mode provides simulated transactions for integration testing. [See External Reference Notes]

FR-02 Payment event validation

- Validate webhook signatures when Razorpay webhooks are used.
- Reject malformed events and duplicate event identifiers.
- Preserve event ordering information where available.
- Record ingestion latency and processing outcomes.

FR-03 State fabric

- Create a state entity for each payment-workload shard or operational domain.
- Store operational metadata, locality preference, replication policy, lifecycle rules, and security policy.
- Maintain versioned state with deterministic transitions.

FR-04 Telemetry ingestion

- Ingest latency, throughput, error rate, queue depth, node health, replication lag, and regional demand signals.

- Aggregate raw telemetry into compact state summaries for AI inference.
- Support synthetic workload traces for reproducible experiments.

FR-05 Agent orchestration

- Monitor Agent detects abnormal or policy-relevant conditions.
- Diagnosis Agent correlates signals and proposes a root-cause hypothesis.
- Planning Agent emits one or more typed action proposals.
- Safety Agent performs a constitutional review and highlights conflicts or risk.

FR-06 Deterministic action safety

- Every proposed action shall pass through typed schema validation.
- Policy rules may allow, deny, modify, or require human approval.
- Decision Verifier shall compare the proposal with current runtime metrics and state.
- No agent may execute raw administrative commands.

FR-07 Runtime execution

- Execute approved actions only through a single gateway.
- Support at least replica creation/removal, traffic shift, partition migration simulation, and rollback.
- Return an action outcome with before/after metrics.

FR-08 Auditability

- Record who/what proposed the action, the evidence, policy result, final action, execution result, and rollback status.
- Support replay of a decision using stored state summary and action artifact.

FR-09 Intent interface

- Accept high-level goals such as “keep payment P95 latency under 250 ms in region X while preserving strong consistency.”
- Compile intent into measurable constraints.

FR-10 Benchmarking

- Run the same workloads with rule-only and agent-assisted orchestration.
- Measure latency, recovery, action safety, token cost, and replica efficiency.

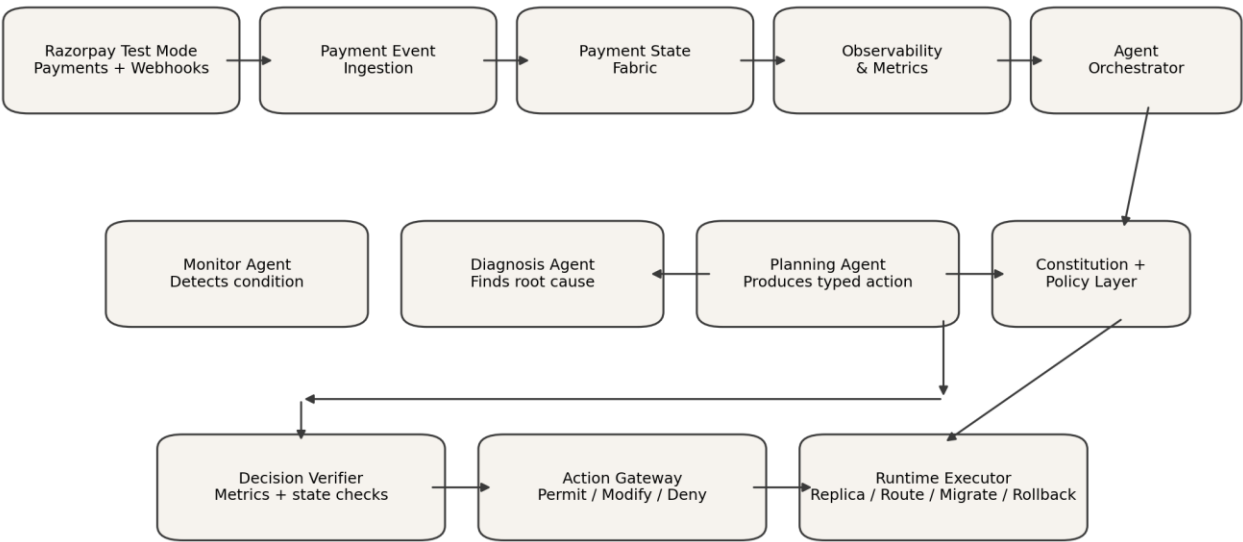
8. Multi-Agent Architecture

Agents provide bounded cognitive functions. They do not own infrastructure authority.

Agent	Inputs	Outputs	Hard boundary
Monitor Agent	Metrics, queue depth, node health, payment event summaries	Condition/event hypothesis	Read-only
Diagnosis Agent	Recent metrics + state history + incident context	Cause hypothesis + evidence references	Read-only
Planning Agent	Diagnosis + intent constraints + current topology	Typed action proposal + rationale	Cannot execute
Safety Agent	Proposed action + ESA Constitution + policy state	Risk classification + policy findings	Cannot execute; may veto

The deterministic layers remain the authority. The Policy Engine validates hard rules; the Decision Verifier confirms the action is appropriate for current state; the Action Gateway is the only execution path. This is consistent with the existing ESA PRD design for typed actions, policy enforcement, risk scoring, rollback, and forensic audit. [See Existing ESA PRD]

9. System Architecture



All executed changes produce an immutable audit event and can trigger rollback.

Figure 1. Payment-angle ESA architecture. The payment event surface provides realistic workflow signals, while the adaptive runtime remains a controlled prototype.

9.1 Layered architecture

Layer	Purpose	Prototype implementation
L1 - Payment Event Surface	Receive and normalize payment events.	Razorpay Test Mode webhooks + synthetic generator
L2 - State Fabric	Maintain operational state and topology metadata.	PostgreSQL/Redis for prototype; optional RocksDB state store
L3 - Agent Orchestration	Detect, diagnose, plan, and review.	Python agents + local SLM via Ollama/llama.cpp
L4 - Deterministic Safety	Validate and gate actions.	FastAPI service + JSON Schema/Pydantic policy engine
L5 - Runtime Executor	Apply controlled topology changes.	Kubernetes simulation or local multi-service cluster
L6 - Observability	Expose metrics, traces, audit lineage.	OpenTelemetry + Prometheus + Grafana

10. Executable State Model

10.1 Payment Workload State Entity

A payment workload entity represents an operational partition or service state rather than a raw payment record. It can carry execution hooks and infrastructure metadata that influence how the runtime places, replicates, and protects the workload.

Field group	Examples
Identity	workload_id, shard_id, version
Operational state	healthy, degraded, overloaded, recovering
Workload signals	rate, p50/p95/p99 latency, error rate, queue depth
Locality	preferred_region, fallback_regions
Replication	min_replicas, max_replicas, consistency_mode
Security	allowed_actions, sensitivity_class, approval_requirement
Lifecycle	create, warm, active, draining, retired

Execution hooks	on_hotspot, on_failure, on_recovery
-----------------	-------------------------------------

10.2 State transition principles

1. Every transition is versioned.
2. An action proposal must identify the state version it was based on.
3. Execution is rejected when the state version is stale beyond the allowed window.
4. Critical transitions require rollback material or a compensating action.
5. The post-action state is measured and stored as evidence.

11. Typed Action Contract

The AI layer outputs structured action objects rather than natural-language commands.

Example action contract:

```
{
  "action": "CREATE_REPLICA",
  "workload_id": "w_1042",
  "target_region": "IN-SOUTH",
  "reason": "P95 latency above policy threshold",
  "expected_effect": {"latency_delta_ms": -80},
  "confidence": 0.91,
  "risk": "LOW",
  "state_version": 481,
  "rollback": {"enabled": true}
}
```

Contract rule

The model never returns shell commands, raw Kubernetes manifests, SQL, or other arbitrary execution text. The runtime accepts only validated action types with bounded parameters.

Action	Use	Default risk
CREATE_REPLICA	Add temporary capacity for a hot workload.	Low/Medium
SHIFT_ROUTE	Move traffic toward a healthy replica/region.	Medium
MIGRATE_PARTITION	Move a hot partition to a more suitable node/region.	Medium/High
THROTTLE_WORKLOAD	Protect the payment processing plane from overload in the prototype.	High
ROLLBACK	Restore prior topology/state configuration.	High but reversible

12. ESA Runtime Constitution

The constitution converts the existing ESA safety concept into payment-infrastructure priorities.

Priority	Rule
1. Safety	Never corrupt state, bypass controls, or execute an unverified mutation.
2. Security/compliance	Respect authentication, authorization, isolation, and logging requirements.
3. Payment correctness	Preserve state consistency and avoid duplicate or contradictory actions.
4. Availability	Prefer recovery and safe failover when correctness is preserved.
5. Reversibility	Prefer actions with rollback or compensating controls.
6. Cost	Optimize capacity only after safety and service constraints are satisfied.
7. Performance	Optimize latency and locality within the above limits.
8. Operator intent	Satisfy stated goals within all hard constraints.

13. Payment Event Integration

Razorpay provides Test Mode and separate Test/Live API keys; Test Mode is intended for integration testing with simulated transactions. Razorpay also provides payment webhooks and recommends validating webhook signatures and designing for asynchronous delivery. [See External Reference Notes]

Integration surface	Prototype use	Safety rule
Test Mode API	Create/fetch test payment context where needed.	Use test credentials only.
Payment webhooks	Consume payment state events such as payment.failed, payment.authorized, payment.captured, order.paid.	Validate signature; deduplicate; audit event.
Synthetic event generator	Generate load, latency, failure, and regional-skew scenarios.	No real customer data.
Runtime simulator	Demonstrate topology and capacity changes.	No control over Razorpay production systems.

Integration boundary

Razorpay Test Mode is the realistic payment-event surface. ESA's adaptive infrastructure layer runs in the project environment; it does not claim access to Razorpay's internal cluster or production routing plane.

14. Data Model

14.1 Core entities

Entity	Key fields
PaymentEvent	event_id, event_type, timestamp, region, payment_method_class, pseudonymous_reference
WorkloadState	workload_id, shard_id, metrics, topology, replication_policy, state_version
AgentDecision	decision_id, agent_id, evidence_refs, proposal, confidence, timestamp
PolicyResult	decision_id, rule_ids, risk, verdict, modifications, approval_required
ActionExecution	execution_id, action_type, parameters, before_state, after_state, outcome
AuditEvent	event_id, actor, action, reason, evidence, policy_result, rollback_ref

14.2 Data minimization

- No raw cardholder data.
- No payment secrets or API key secrets in state prompts.
- Pseudonymize identifiers before AI inference.
- Aggregate metrics before sending them to the SLM.
- Retain only the evidence required for debugging and research evaluation.

15. Technology Stack

Area	Recommended stack	Why
Primary runtime	Rust + Tokio	Concurrency, memory safety, strong systems fit.
Prototype APIs	Python + FastAPI	Fast iteration for orchestration and policy services.
Agent framework	Custom typed agent loop or lightweight orchestration layer	Keep execution deterministic and inspectable.
SLM	Qwen/Phi/TinyLlama-class local model	Low-latency local reasoning and controllable cost.
Model serving	Ollama or llama.cpp	Simple local inference for prototype.
State store	PostgreSQL + Redis	Durable metadata plus low-latency counters/cache.
Optional state engine	RocksDB	Research path for write-heavy embedded state.
Messaging	NATS	Lightweight event bus for telemetry and runtime events.
Sandbox	WASM + Wasmtime	Bounded execution for executable entities.
Consensus	Raft library / prototype coordination	Understandable coordination model for research demo.
Containers	Docker	Reproducible local and CI environment.
Runtime platform	Kubernetes (kind/minikube for demo)	Demonstrate scaling, failover, and topology changes.
Observability	OpenTelemetry + Prometheus + Grafana	Traces, metrics, dashboards, action lineage.
Security	mTLS + SPIRE-like workload identity + Vault/Secrets	Least privilege and secret management.

Schema validation	Pydantic / JSON Schema	Typed action contracts and strict validation.
CI/CD	GitHub Actions	Automated test and reproducible builds.
Payment integration	Razorpay Test Mode APIs + Webhooks	Realistic payment event integration without real money.

16. Infrastructure Topology for the Prototype

The prototype should be small enough to run locally or on a low-cost cloud environment while still demonstrating distributed behavior.

Component	Count target	Notes
API/Event Gateway	2 replicas	Receives webhooks and normalizes events.
State nodes	3 nodes	Replicated metadata and simulated workload state.
Agent service	1 service	Hosts the four bounded agents.
Policy service	2 replicas	Stateless deterministic checks.
Runtime workers	3-6 workers	Scale in/out during the demo.
Metrics stack	1 stack	Prometheus + Grafana + OpenTelemetry collector.
Message bus	3-node NATS optional	Use single-node for simplest Buildathon demo if needed.

17. Security Architecture

- Webhook signature validation before event acceptance.
- Separate Test Mode secrets from all development and production credentials.
- Least-privilege service identities.
- mTLS between runtime services where practical.
- Agent isolation from direct infrastructure credentials.
- Policy Engine as a mandatory authorization boundary.
- Signed or integrity-protected action artifacts.
- Immutable or append-only audit storage for evaluation.
- Human approval for high-risk actions.
- Rollback snapshots before major topology mutations.

Razorpay specifically documents webhook signature validation and asynchronous delivery/retry behavior; the prototype should model these realities rather than assuming all events arrive exactly once or in a perfectly synchronous sequence. [See External Reference Notes]

18. Reliability and Failure Handling

Failure	Detection	Recovery behavior
Duplicate webhook	Duplicate event ID / idempotency check	Ignore duplicate execution; preserve audit trail.
Out-of-order event	Event timestamp/sequence mismatch	Store, reconcile, or defer transition.
Agent unavailable	Service health check	Fall back to deterministic rules.
LLM timeout	Inference deadline exceeded	Use cached decision or rule-only policy.
Stale state version	Version mismatch at gateway	Reject action and require replanning.
Runtime mutation failure	Execution error + health metrics	Rollback or compensating action.
Policy violation	Policy Engine verdict	Deny and record explanation.

19. Observability and Explainability

Every autonomous action should answer five questions: What happened? What did the agents believe? What action was proposed? Why was it allowed or denied? What happened after execution?

Dashboard	Key panels
-----------	------------

Payment health	Throughput, P95/P99 latency, failure ratio, queue depth.
Topology	Node state, replicas, locality, current workload distribution.
Agent reasoning summary	Condition, evidence references, confidence, proposed action.
Policy decisions	Allowed, modified, denied, approval-required counts.
Action outcomes	Before/after metrics, success, rollback, duration.
AI cost	Token usage, model calls, cache hit rate, decision latency.

20. Key Product Metrics and Benchmarks

Metric	Definition	Target use
P95 payment-workload latency	95th percentile processing/serving latency in controlled workload.	Show performance impact.
Recovery time	Time from failure detection to stable recovered state.	Compare adaptive vs rule-only.
Hotspot resolution time	Time from hotspot detection to latency stabilization.	Show topology adaptation.
Replica efficiency	Capacity added relative to workload improvement.	Measure unnecessary scaling.
Action safety rate	Allowed actions that satisfy all policy and state checks.	Measure governance quality.
Policy violation rate	Unsafe proposals blocked or modified by policy.	Demonstrate guardrails.
Rollback success rate	Successful reversions after failed mutations.	Demonstrate reversibility.
AI decision latency	Inference + verification time per decision.	Measure operational overhead.
Token cost per event	Model tokens consumed per orchestration cycle.	Measure cost efficiency.

21. Experimental Design

21.1 Baselines

- Rule-only orchestration: deterministic thresholds without agent reasoning.
- ESA with SLM and deterministic policy layer.
- Optional manual baseline: no automatic topology mutation.

21.2 Workload matrix

Workload	Variable
Steady	Stable rate, stable latency.
Burst	Sudden 3-5x load increase.
Regional skew	One region receives disproportionate traffic.
Node failure	One runtime node removed or degraded.
Queue buildup	Event processing rate becomes slower than arrival rate.
Mixed incident	Burst + one unhealthy node + increased latency.

21.3 Ablation studies

- Rules only vs agents + rules.
- Single-agent planner vs multi-agent pipeline.
- With vs without decision caching.
- With vs without compact state summarization.
- With vs without rollback enforcement.

22. Primary Demo Scenario

The flagship demo should be a controlled payment-workload hotspot followed by safe recovery.

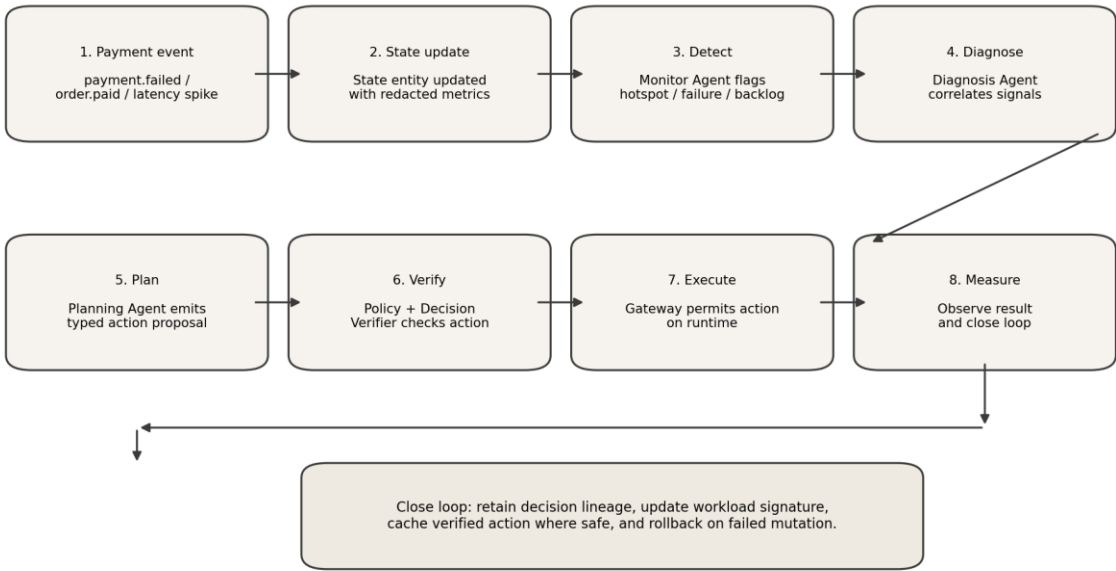


Figure 2. Recommended end-to-end demo workflow.

- 1. Generate payment traffic in Razorpay Test Mode and/or synthetic mode.
- 2. Introduce regional skew or a controlled workload spike.
- 3. Observe rising P95 latency and queue pressure.
- 4. Monitor Agent raises the condition.
- 5. Diagnosis Agent correlates latency, queue, replica, and health signals.
- 6. Planning Agent proposes CREATE_REPLICA or SHIFT_ROUTE.
- 7. Safety Agent and deterministic Policy Engine evaluate the action.
- 8. Decision Verifier checks state version, metrics, and limits.
- 9. Action Gateway permits the bounded mutation.
- 10. Runtime executes the change and metrics improve.
- 11. Audit dashboard shows the complete decision lineage.
- 12. Optionally inject an execution failure and demonstrate rollback.

23. MVP Scope for the Razorpay Buildathon

The existing ESA PRD is intentionally broader than the Buildathon submission. For the Buildathon, the MVP should focus on one complete, measurable loop rather than implementing every planned research feature.

Buildathon MVP	Defer to later ESA research
Payment event ingestion	Full custom storage engine
3-4 bounded agents	Complex multi-cluster consensus research
Policy Engine + Action Gateway	Complete WASM entity ecosystem
2-3 typed runtime actions	Advanced predictive modeling
One distributed hotspot scenario	Full global placement optimization
Audit + rollback	Production-grade secrets platform
Benchmark vs rule-only baseline	Advanced quantum extensions

Success condition

A small but convincing system that works end to end is stronger than a large architecture with incomplete execution. The Buildathon demo should prioritize evidence of action, safety, and measurable value.

24. Buildathon Demo Scope and September 5, 2026 Freeze

Purpose: This section defines the smallest end-to-end ESA payment-infrastructure prototype that must be implemented and demo-ready by 5 September 2026. It is an internal delivery target for the Razorpay Buildathon submission and is intentionally narrower than the full six-month ESA roadmap.

24.1 Demo Objective

Demonstrate one complete, observable, and measurable loop in which a controlled payment-workload condition is detected, analyzed by bounded agents, converted into a typed action, deterministically verified, safely executed on the prototype runtime, and recorded in an auditable decision trail.

24.2 Mandatory Demo Scenario

The primary scenario is a regional payment-workload hotspot. A synthetic payment workload and/or Razorpay Test Mode event stream creates a controlled load increase or regional skew. The runtime detects rising P95 latency and queue pressure, the agent system diagnoses the hotspot, proposes a bounded topology action, and the safety layer approves or rejects it before execution.

Step	Demo behavior	Evidence shown
1	Generate payment workload using synthetic events and, where practical, Razorpay Test Mode events.	Event stream and workload dashboard.
2	Inject regional skew or a controlled 3x-5x workload burst.	Traffic/load panel and scenario marker.
3	Detect degraded P95 latency, queue pressure, or error-rate change.	Live metrics before intervention.
4	Monitor Agent identifies the condition.	Condition event with timestamp.
5	Diagnosis Agent identifies likely hotspot/root cause from recent state and metrics.	Evidence references and diagnosis summary.
6	Planning Agent proposes CREATE_REPLICA and/or SHIFT_ROUTE using a typed contract.	Structured action proposal.
7	Safety Agent + Policy Engine + Decision Verifier evaluate the proposal.	Risk score, policy verdict, and state-version check.
8	Action Gateway permits, modifies, or blocks the action.	Gateway decision and audit entry.
9	Runtime executor applies the approved change.	Replica/topology/runtime state change.
10	Workload recovers or stabilizes.	After-metrics showing latency/queue improvement.
11	Audit dashboard shows the full decision lineage.	Event -> evidence -> policy -> action -> outcome.
12	Optional failure injection triggers rollback.	Rollback event and restored state.

24.3 Demo Architecture Scope

Component	Required for September 5 demo	Implementation boundary
Payment Event Surface	Yes	Synthetic payment-event generator is mandatory; Razorpay Test Mode webhook integration is preferred if stable.
Workload State Store	Yes	PostgreSQL + Redis is sufficient for the demo; no custom storage engine required.

Monitor Agent	Yes	Read-only metric/queue/health observation.
Diagnosis Agent	Yes	Correlates recent metrics and state history; returns evidence references.
Planning Agent	Yes	Returns only typed actions such as CREATE_REPLICA or SHIFT_ROUTE.
Safety Agent	Yes	Checks proposal against the ESA Constitution and produces a risk/policy result.
Policy Engine	Yes	Deterministic allow/deny/modify rules.
Decision Verifier	Yes	Validates state version, current metrics, permissions, and action limits.
Action Gateway	Yes	Single execution path for agent-approved mutations.
Runtime Executor	Yes	Local Kubernetes/minikube/kind or equivalent multi-service simulation.
Observability	Yes	OpenTelemetry + Prometheus + Grafana, with a lightweight audit view.
Rollback Manager	Yes	At least one demonstrated recovery path after an injected execution failure.
WASM entity runtime	No	Defer beyond the Buildathon demo unless already stable.
Raft/advanced distributed consensus	No	Use the minimum coordination mechanism required for the demo; defer full research implementation.
Predictive scaling model	No	Defer; rule/metric-triggered adaptation is sufficient for the first demo.

24.4 Definition of Done - September 5

The Buildathon demo is considered complete only when all mandatory items below pass in a clean end-to-end run:

- A controlled payment workload can be generated repeatedly and produces deterministic test data.
- At least one workload condition (regional hotspot/burst) is detected automatically.
- The four bounded agents produce an understandable diagnosis, typed action proposal, and safety decision.
- No agent can directly execute arbitrary SQL, shell commands, kubectl commands, or raw infrastructure instructions.
- At least two typed actions are implemented end to end: CREATE_REPLICA and SHIFT_ROUTE, or CREATE_REPLICA and ROLLBACK.
- The Policy Engine can demonstrate at least one allowed action, one blocked action, and one stale-state rejection.
- The runtime visibly changes capacity/topology and the workload metrics respond to the change.
- The audit trail records the condition, evidence references, proposal, policy result, execution result, and timestamp.
- At least one injected execution failure produces a successful rollback or compensating action.
- A before/after benchmark is recorded against a rule-only or static baseline.
- The public GitHub repository contains runnable setup instructions and a reproducible demo path.
- A 5-minute pitch can complete the full scenario without manual code edits during the presentation.

24.5 Metrics Required for the Demo

Metric	Minimum evidence
P95 workload latency	Before vs after the adaptive action.
Hotspot resolution time	Condition detection timestamp to stabilization.
Queue depth / drain time	Peak queue and recovery trend.
Action latency	Proposal -> verification -> execution duration.
Policy safety outcome	Allowed / modified / denied / stale-state counts.
Rollback success	Whether the injected failure returned the runtime to the previous valid state.

AI inference cost	Model calls, approximate tokens, and cache hits for the demo run.
-------------------	---

24.6 September 5 Delivery Sprint

Because the current date is 23 August 2026, the target leaves approximately 13 days to reach the demo freeze. The sprint is intentionally focused on one vertical slice; any feature outside this slice is optional and must not delay the working demo.

Window	Primary goal	Exit condition
23-24 Aug	Freeze architecture and scaffold services.	Repository structure, event schema, state schema, action contracts, and runnable local stack exist.
25-26 Aug	Implement payment event + workload simulator and state updates.	Synthetic/Test Mode events flow into observable workload state.
27-28 Aug	Implement Monitor + Diagnosis agents.	Hotspot condition is detected and explained from live metrics/state.
29-30 Aug	Implement Planning + Safety agents and typed actions.	Typed proposal reaches deterministic policy checks; arbitrary execution is impossible.
31 Aug-1 Sep	Implement Action Gateway + runtime mutation.	CREATE_REPLICA and/or SHIFT_ROUTE execute successfully in the prototype runtime.
2 Sep	Implement audit trail + rollback + failure injection.	One full failure-and-recovery scenario works end to end.
3 Sep	Benchmark against rule-only/static baseline.	Before/after metrics and at least one comparative result captured.
4 Sep	Stabilize, document, record pitch, clean repository.	Fresh-machine/local clean run succeeds; video and architecture are ready.
5 Sep	Demo freeze.	No unfinished mandatory feature remains; final demo path is reproducible and presentation-ready.

24.7 Stretch Features - Only After the Mandatory Demo Works

- Razorpay Test Mode webhook ingestion with signature validation and duplicate-event handling.
- A third runtime action such as MIGRATE_PARTITION or THROTTLE_WORKLOAD.
- Decision caching and token-budget optimization comparison.
- A richer topology visualization and action-lineage replay view.
- Multi-node NATS deployment and more realistic failure injection.
- Executable WASM state-entity prototype.

24.8 Demo Freeze Rule

No new architectural subsystem should be added after the 2 September integration checkpoint unless it directly fixes a blocker in the mandatory demo path. The priority order is: reliability of the end-to-end workflow, deterministic safety, measurable evidence, reproducibility, and only then feature breadth.

25. Six-Month Implementation Roadmap

Month	Focus	Exit criteria
1	Payment event surface + workload simulator	Test Mode/synthetic events ingest reliably; baseline dashboards.
2	State fabric + deterministic runtime	Executable state entities and controlled action API working.
3	Agent layer	Monitor, Diagnosis, Planning, and Safety agents produce typed proposals.
4	Safety and governance	Policy Engine, Decision Verifier, Action Gateway, audit trail, rollback.
5	Distributed adaptation	Replica, routing, and failure-recovery demos working on a local cluster.

6	Benchmark + pitch	Ablations, metrics, public repo cleanup, 5-minute demo, architecture diagram.
---	-------------------	---

26. Acceptance Criteria

ID	Acceptance criterion
AC-01	A payment event can trigger an observable state update.
AC-02	The agents can detect and diagnose at least one workload condition.
AC-03	The Planning Agent outputs a valid typed action.
AC-04	Unsafe or stale actions are deterministically blocked.
AC-05	A safe action changes the runtime prototype state.
AC-06	The system records complete decision lineage.
AC-07	At least one rollback scenario succeeds.
AC-08	Adaptive orchestration is benchmarked against rule-only control.
AC-09	No secret or sensitive cardholder data is exposed to the model.
AC-10	The five-minute pitch demonstrates a complete end-to-end workflow.

27. Risks and Mitigations

Risk	Impact	Mitigation
Scope becomes too large	High	Freeze MVP around one end-to-end hotspot/recovery workflow.
Agent hallucination	High	Typed contracts, deterministic policies, state-version checks.
Payment integration delays	Medium	Keep a synthetic event generator as a first-class path.
Weak benchmark	High	Define metrics and workload matrix before implementation.
LLM overhead	Medium	Local SLM, summarization, routing, cache, strict token budgets.
Unsafe autonomous mutation	High	Single Action Gateway + Policy Engine + Decision Verifier + rollback.
Reviewer misunderstands concept	Medium	Use one concrete payment hotspot story before architecture depth.

28. Pitch Narrative

1. Problem - Payment infrastructure changes faster than static operational rules can safely adapt.
2. Insight - Infrastructure needs reasoning, but unrestricted AI control is unsafe.
3. ESA - State becomes executable and policy-aware; agents propose bounded changes.
4. Demo - A payment workload hotspot triggers diagnosis, planning, policy verification, runtime mutation, and measured recovery.
5. Proof - Compare against rule-only orchestration and show latency/recovery/safety metrics.
6. Why it matters - The project turns AI from an unrestricted operator into a bounded infrastructure decision layer.

29. What Not to Claim

- Do not claim ESA controls Razorpay production infrastructure.
- Do not claim the prototype is equivalent to Razorpay internal architecture.
- Do not claim autonomous decisions are always correct.
- Do not claim zero fraud, zero downtime, zero risk, or “unhackable” infrastructure.
- Do not claim that Open Track is easier than the predefined tracks.
- Do not present the SLM as having unrestricted infrastructure access.

30. Relationship to the Existing ESA PRD

This document preserves the core ESA architecture and safety philosophy from the existing PRD while narrowing the Buildathon narrative around payment infrastructure. The original PRD defines executable state entities, AI-governed runtime control, deterministic safety, typed action contracts, rollback, auditability, intent-driven interaction, token-cost optimization, and distributed runtime layers. [See Existing ESA PRD]

The payment-angle PRD adds a concrete operating context, payment event integration, payment workload state modeling, payment-specific scenarios, a bounded multi-agent design, a Buildathon MVP boundary, and a six-month execution plan. It does not replace the longer-term ESA research roadmap.

31. External Reference Notes

The following official Razorpay documentation was used only to ground the payment integration boundary and event model. It does not imply access to or knowledge of Razorpay internal infrastructure.

Razorpay API Reference Guide: <https://razorpay.com/docs/api/> - REST API structure and Test Mode/API testing context.

Razorpay Quickstart: <https://razorpay.com/docs/payments/quickstart/> - API keys, Test/Live modes, webhooks, and payment integration workflow.

About Webhooks: <https://razorpay.com/docs/webhooks/> - Payment-related event notifications and server-side webhook usage.

Payments Webhook Events: <https://razorpay.com/docs/webhooks/payments/> - Payment event lifecycle and webhook event semantics.

Validate and Test Webhooks: <https://razorpay.com/docs/webhooks/validate-test/> - Webhook signature validation and Test Mode testing.

Webhook Best Practices: <https://razorpay.com/docs/webhooks/best-practices/> - Retry, timeout, asynchronous delivery, and webhook reliability behavior.

Test and Live Modes: <https://razorpay.com/docs/payments/dashboard/test-live-modes/> - Separation of Test and Live environments.

32. Final Product Definition

One-sentence definition

ESA is a policy-bounded multi-agent runtime that converts payment-workload signals into verified, typed infrastructure actions, executes them through a controlled gateway, and measures every outcome.

Buildathon-ready framing

Open Track | Payment infrastructure resilience | Executable state + bounded agents + deterministic safety | Working prototype with measurable improvement over rule-only orchestration